

PROGRAMACIÓN AVANZADA

Interfaz de Usuario (GUI)
Swing – Mas componentes



Ing. Matias Macrino

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JCheckbox:

-> Dicho componente permite seleccionar entre distintas opciones posibles.
Para poder instanciar un objeto de dicho componente, se puede hacer mediante su constructor:

```
JCheckBox chkEspaniol = new JCheckBox("Español");  
panel.add(chkEspaniol);
```

-> Posee un método que nos indica si el check fue seleccionado. **Boolean : isSelected()**

```
chkEspaniol.isSelected();
```

-> También posee un método para seleccionar o no el check **setSelected(boolean)**.

```
chkEspaniol.setSelected(true);
```

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JCheckbox:

-> Manejo de eventos, puede estar asociado mediante el uso de “**ActionListener**”, aunque se recomienda utilizar “**ChangeListener**”. El mismo se debe utilizar agregando el escuchador con el método “**addChangeListener**”.

Antes cualquier cambio de estado del componente, se dispara el método “**stateChanged**”, el cual recibe un **ChangeEvent**, que nos permite conocer que sucedió.

```
chkEspaniol.addChangeListener(new ChangeListener () {  
    @Override  
    public void stateChanged(ChangeEvent e) {  
        [CODIGO.....]  
    }  
});
```

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JRadioButton:

- ▶ Los Botones de Radio son grupos de botones en los que, por convención, sólo uno puede estar seleccionado. Sólo admite dos estados posibles: seleccionado o no seleccionado (éste último por defecto).
- ▶ Están soportados en la librería Swing, mediante el uso de las clases **JRadioButton** y **ButtonGroup**.
- ▶ Cada vez que el usuario pulsa un botón de radio, se dispara un **action event**.
- ▶ Para cada grupo lógico de botones de radio, se necesita crear un ejemplar de la clase **ButtonGroup**.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JRadioButton:

- ▶ Los métodos más utilizados de la clase **JRadioButton** son **setMnemonic**, **addChangeListener**, **setSelected** y **isSelected**.
- ▶ Los métodos más utilizados de la clase **ButtonGroup** son **add** y **remove**.
- ▶ Para crear un botón de radio (constructor con parámetros):
JRadioButton(String text, Icon icon, boolean selected)
- ▶ Para crear un grupo de botones (constructor sin parámetros):
ButtonGroup()

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JComboBox:

Este componente permite presentar una lista posible de respuestas de las cuales el usuario seleccionara una de ellas.

Esta formado por dos partes:

- Encabezado: titulo.

- Lista de opciones.

En Runtime, solo se visualiza el encabezado y mediante una flecha se despliegan las opciones.

El método `getSelectedIndex()` retorna la posición del objeto marcado por el usuario.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

COMPONENTES

¿Cómo llenar un combo?

El mismo se puede llenar desde su constructor, como se ve en el siguiente ejemplo:

```
String[] elementos = {"", "Bs.As", "Santa Fe", "Cordoba"};
```

```
JComboBox combo = new JComboBox(elementos);
```

Otra de las maneras es utilizar el método `addItem(Object object);`

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

COMPONENTES

¿Cómo llenar un combo?

También se puede agregar elementos al combo mediante la creación de un objeto.

```
public class Pais {  
    private long id;  
    private String nombrePais;  
    //Constructores, getters & setters  
    @Override  
    public String toString() {  
        return "Pais [nombrePais=" + nombrePais + "];"  
    }  
}
```

Construir objeto y sobrescribir el método **toString()**

```
Pais francia = new Pais();  
francia.setId(112234);  
francia.setNombrePais("Francia");  
combo.addItem(francia);
```

Instanciar objeto y agregarlo al JComboBox mediante **addItem(Object object)**

```
Pais paisSeleccionado = (Pais) combo.getSelectedItem();  
System.out.println("El id del pais es " + paisSeleccionado.getId());  
System.out.println("El nombre del pais es " + paisSeleccionado.getNombrePais());
```

Mediante el método **getSelectedItem()**, se obtiene el objeto seleccionado del combo. El mismo retorna un **Object**

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JComboBox:

-> Manejo de eventos, puede estar asociado mediante el uso de “**ActionListener**”, aunque se recomienda utilizar “**ItemListener**”. El mismo se debe utilizar agregando el escuchador con el método “**addItemListener**”.

Antes cualquier cambio de estado del componente, se dispara el método “**itemStateChanged**”, el cual recibe un **ItemEvent**, que nos permite conocer que sucedió.

```
JComboBox combo = new JComboBox();
combo.addItemListener(new ItemListener () {
    @Override
    public void ItemStateChanged(ItemEvent e) {
        [CODIGO.....]
    }
});
```

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JList:

-> El componente **JList** provee una lista con ítems de objetos, donde uno o más pueden ser seleccionados por el usuario.

- Ejemplo: `JList lista = new JList(arrayObjetos);`

-> La clase **JList** soporta dos formas de selección:

1 - **Single selection list:** solo puede seleccionarse un objeto a la vez

2 - **Multiple selection list:** puede seleccionarse varios objetos a la vez, continuos y discontinuos.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JList:

-> La forma de selección se indica mediante las constantes static de la clase **ListSelectionModel**, con el metodo **setSelectionMode**. Ejemplo:

- ✓ ListSelectionModel.SINGLE_SELECTION: selección simple
- ✓ ListSelectionModel.SINGLE_INTERVAL_SELECTION: selección múltiple pero solo continúa – tecla Shift
- ✓ ListSelectionModel.MULTIPLE_INTERVAL_SELECTION: selección múltiple y discontinúa – tecla Control

-> El método **getSelectIndex()** sobre una lista simple, devuelve la posición del objeto seleccionado. Si no hay ninguno seleccionado, éste retorna -1.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JList:

-> El método **getSelectedValues()** sobre una lista múltiple, devuelve un array con los objetos seleccionados.

-> Manejo de eventos:

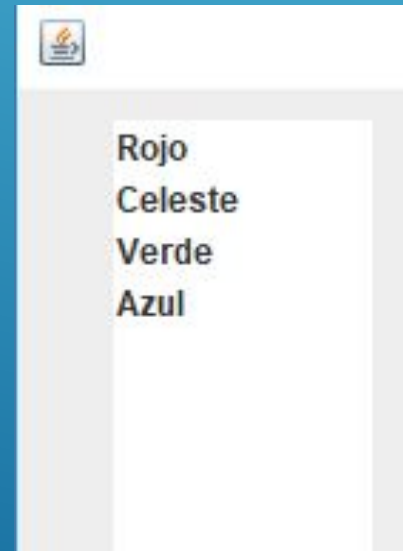
- ✓ Los eventos asociados a este componente son administrados por la interfaz **ListSelectionListener**.
- ✓ Mediante el método **addListSelectionListener()** se registra el disparador vinculado a la lista.
- ✓ Se debe implementar el método **valueChanged()**; el cual recibe como parámetro un objeto de la clase **ListSelectionEvent**.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JList:

-> Ejemplo:

```
String[] colores = {"Rojo", "Celeste", "Verde", "Azul"};  
list = new JList(colores);  
list.setBounds(34, 11, 91, 222);  
contentPane.add(list);
```



APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

JList - DefaultListModel:

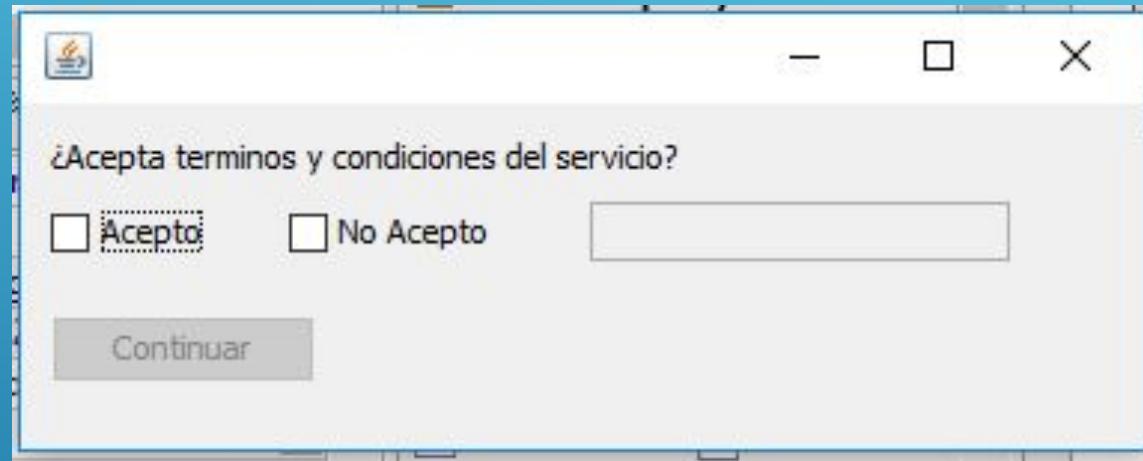
-> Otra de las formas que tendremos para completar una JList es utilizar un objeto del tipo “**DefaultListModel**”. Lo que nos permite es manejar de manera independiente la lista del componente.

-> Uso del **DefaultListModel**:

- ✓ Instanciar un objeto, con el constructor:
`DefaultListModel modelo = new DefaultListModel();`
- ✓ Agregarlo al componente JList, de las siguientes dos formas:
 - a- `JList lista = new JList(modelo);`
 - b- `lista.setModel(modelo);`
- ✓ Agregar elemento al modelo con:
`modelo.addElement("Bs. As");`
`modelo.addElement("Cordoba");`

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

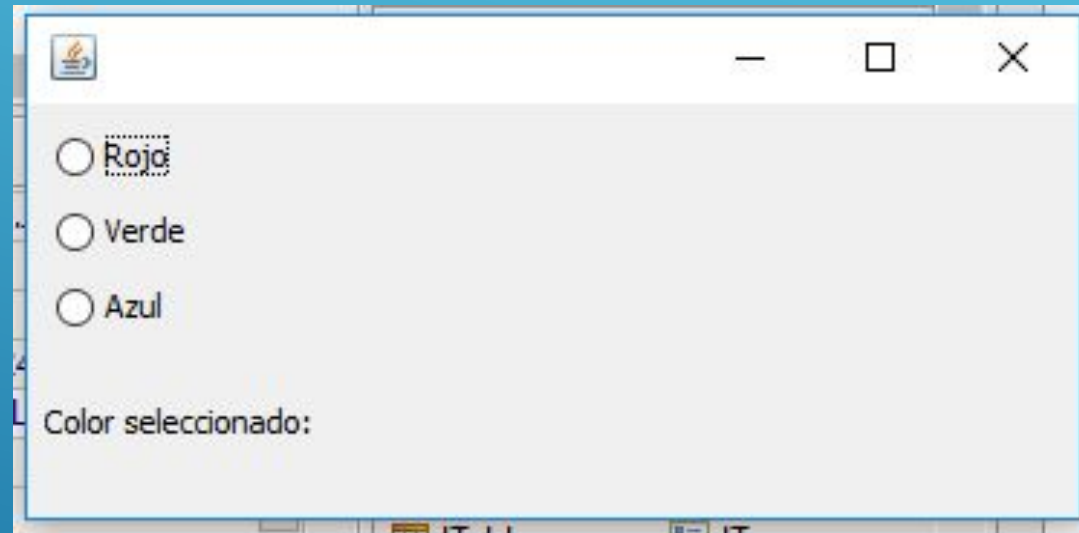
Jcheckbox - Practica:



- Al seleccionar el check "Acepto", habilitar el botón.
- Al seleccionar el check "No Acepto", habilitar el JTextField y el botón.
- El botón debe de salir de la aplicación.
- El botón tiene que validar si se ingreso el motivo de no aceptar antes de salir de la aplicación.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

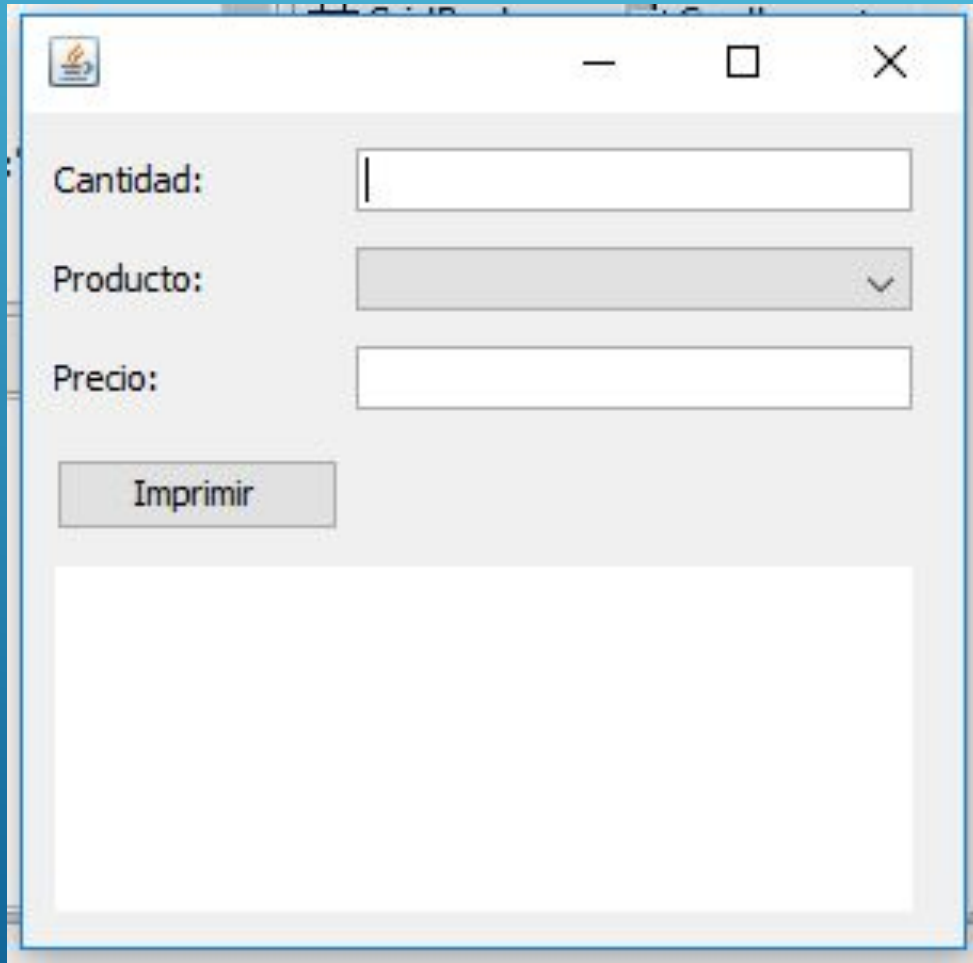
JRadioButton – Practica:



- Al ser un JRadioButton se permite la selección de una sola opción.
- Al seleccionar el color, se tiene que mostrar el nombre seleccionado y el JLabel debe de cambiar al color seleccionado.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Jlist/JComboBox - Practica:



The screenshot shows a Java Swing window titled "Practica" with a standard Mac OS X title bar (red, yellow, and green buttons). The window contains a form with three input fields: "Cantidad:" (a text box with a vertical cursor), "Producto:" (a JComboBox with a dropdown arrow), and "Precio:" (a text box). Below these fields is a button labeled "Imprimir". At the bottom of the window is a large, empty rectangular area, likely intended for a JList or JTextArea.

- El botón imprimir trasladará los datos ingresados al Jlist.
- Utilizar DefaultListModel.
- Realizar la aplicación mediante objetos.
- En el Jlist se visualizara producto, cantidad y precio total.